

**¿Cuánto se puede creer una diferencia? Validez de las tareas y variación entre corridas
en una evaluación exploratoria de un agente de código con Gemma 4**

Seis cambios de configuración, ninguna mejora establecida, y un fallo de herramienta que desapareció sin cambiar qué tareas se resolvieron.

Cristóbal Herrera Jara

Resumen

Ajustamos la configuración de un agente de código con Gemma 4 para la competición Gemma 4 Developer Agent e informamos, en conteos agregados, lo que medimos para leer nuestras propias comparaciones. En seis cambios de configuración no establecimos ninguna mejora; el mayor aumento observado no se replicó, y eso no es evidencia de que no haya efecto. Primero, en el entorno del notebook de la competición, 71 de las 129 tareas públicas separan el parche de referencia de la ausencia de parche; al instalar tres paquetes que solo usan las pruebas son 103. Segundo, entre corridas idénticas cambiaron de resultado 1 a 2 tareas individuales, mientras el total cambió en 0 a 1 dentro de cada par; un mismo archivo local enviado dos veces obtuvo 0,06 y 0,05 en la tabla pública, sin verificar que los dos archivos almacenados fueran idénticos. Tercero, la herramienta de edición se llamó sin un argumento obligatorio en 107 de 134 y 49 de 75 llamadas en dos corridas base, y en 0 de 26 en una corrida con una línea de instrucción añadida; las tareas resueltas fueron las mismas 7 de 30 que en la segunda corrida base. Cuarto, de 23 tareas nunca resueltas en tres corridas, una revisión retrospectiva juzgó resolubles 12 con el enunciado y el repositorio; 10 de ellas no llegaron a una edición pertinente en al menos dos de las tres corridas.

¿Cuánto se puede creer una diferencia? Validez de las tareas y variación entre corridas en una evaluación exploratoria de un agente de código con Gemma 4

1. Introducción

Quien participa en la competición entrega una configuración para un modelo y un arnés fijos, y recibe la fracción de tareas ocultas cuyo parche pasa pruebas ocultas. El desarrollo se hace sobre 129 tareas públicas. Con unas decenas de tareas por corrida y un agente estocástico, un cambio de 1 o 2 tareas resueltas es a la vez el efecto que se busca y el que produce el azar.

Hacemos una sola pregunta:

En este contexto, ¿qué diferencia entre dos configuraciones de un agente puede distinguirse de repetir la misma configuración?

La contribución es lo que medimos para leer nuestras propias comparaciones:

1. Qué tareas públicas discriminan en el notebook, con la causa confirmada por intervención en 31 de las 58 que no discriminan (§ 4.1).
2. La variación entre corridas de una configuración fija: resultados por tarea, totales y categorías de fallo (§ 4.2).
3. Seis intervenciones con predicciones anotadas antes de cada corrida; en una, un mecanismo de fallo desapareció y las tareas resueltas no cambiaron (§ 4.3).
4. Dónde se pierde el agente en las tareas nunca resueltas, con un juicio retrospectivo de cuántas eran resolubles (§ 4.4).

La evidencia viene de tres fuentes, separadas: controles sin el modelo (§ 4.1), corridas del agente con Gemma 4 (§ 4.2 a § 4.4) y el juicio de un revisor (§ 4.4).

2. Trabajos relacionados

SWE-bench plantea incidencias reales de GitHub cuya solución es un parche al código (Jimenez et al., 2024). Auditorías posteriores informan de pruebas débiles y de soluciones dadas en la incidencia o en sus comentarios (Aleithan et al., 2024), y parches contados como correctos que no pasan las pruebas de los desarrolladores (Wang et al., 2026). Zhu et al. (2025) informan que muchos benchmarks de agentes tienen problemas en el montaje de las tareas o en el diseño de la recompensa. Dos notebooks públicos auditaron antes las tareas públicas de esta competición: Xodarev (2026) informa 119 de 129 sólidas con un evaluador reimplementado y entornos reconstruidos, y Gluzdov (2026), 114 tras reparar solo dependencias, con la lógica oficial de puntuación sin cambios. No los reproducimos; los tres conteos difieren en entorno, criterio de control y evaluador.

En SWE-bench Verified, la tasa de acierto de una sola corrida varió en 2,2 a 6,0 puntos porcentuales según la corrida elegida, con tres modelos y dos andamiajes (Bjarnason et al., 2026). Miller (2024) da fórmulas para analizar y planificar una evaluación. En SWE-bench Verified, las mejoras pueden reflejar en parte memorización (Liang et al., 2026). Escribir las predicciones antes de los datos sigue el espíritu del prerregistro (Nosek et al., 2018).

Una intervención añade texto de instrucción. Gloaguen et al. (2026) hallaron que los archivos de contexto del repositorio no mejoraron en general el éxito en las tareas, aunque los agentes siguieron sus instrucciones.

3. Contexto y método

Modelo y arnés. La competición fija el modelo, `gemma-4-31b-it-qat-w4a16-ct`, y un arnés con herramientas para ejecutar comandos, leer, buscar y editar archivos, y entregar un parche. El conjunto oculto tiene unas 120 tareas de repositorios privados; las 129 públicas vienen de cuatro repositorios de código abierto.

Línea base. El kit de inicio oficial sin adaptadores, 8192 tokens de salida, razonamiento del modelo apagado y un presupuesto por tarea de 4 minutos, 40 llamadas a

herramientas, 100 turnos y 60 segundos por comando. El muestreo usa temperatura 0,2 sin semilla.

Corridas. Una *corrida* es una pasada de una configuración sobre una lista fija de tareas, en un notebook de Kaggle con cuatro GPU L4. El conjunto de medición tiene 30 tareas: 15 de un repositorio (rich), usadas durante todo el desarrollo, y 15 de otro (fastapi). Las de fastapi solo eran inéditas en la primera de las tres corridas de 30 tareas. Las sesiones anteriores usaron esas 15 tareas de rich, o un conjunto de 16 que las contiene. Las 30 discriminan (§ 4.1). Las trazas completas se guardaron desde la primera corrida de 30 tareas.

Regla del ciclo. Antes de cada corrida anotamos una predicción en conteos y el resultado que la refutaría. Cada cambio de § 4.3 altera una sola cosa. Como regla operativa, un cambio contaba como mejora solo con una ganancia neta de al menos tres tareas resueltas; esa regla no tiene detrás prueba estadística ni análisis de potencia.

Desviaciones, declaradas. Prerregistramos otro diseño: reservar un repositorio y comparar corridas repetidas con la prueba de McNemar (Dietterich, 1998). Nunca lo ejecutamos: el repositorio reservado se usó para desarrollo y la variación se midió contando tareas. Todos los resultados son, por tanto, exploratorios. El registro de predicciones estuvo fuera del control de versiones en este periodo, y cinco entradas llevan horas estimadas.

4. Resultados

4.1 El instrumento: qué tareas públicas pueden medir algo

Una tarea *discrimina* si sus pruebas fallan sin parche y pasan con el parche de referencia. Ejecutamos ambos controles en las 129 tareas públicas, en el notebook y sin el modelo (Tabla 1).

Tabla 1*Tareas públicas que discriminan, por repositorio y entorno*

Repositorio	Tareas	Tal como viene	Con tres paquetes de pruebas instalados
fastapi	67	29	60
rich	48	42	43
requests y httpx	14	0	0
Total	129	71	103

Nota. Cada celda cuenta tareas cuyas pruebas fallan sin parche y pasan con el parche de referencia. Una corrida de control por celda; sin modelo.

Tal como viene, 58 tareas no discriminan: en 55 el parche de referencia no pasa y 3 pasan sin parche. En 34 de las 55 las pruebas no se ejecutaron porque faltaba un paquete que solo ellas usan. Con los paquetes instalados, 31 de esas 34 discriminan. Otra tarea de rich pasó a discriminar sin causa identificada, y no se perdió ninguna de las 71 originales. Siguen sin discriminar 26 tareas: 23 fallan con el parche de referencia, causa no examinada, y 3 pasan sin parche.

De las 58 tareas (45 % de 129) que no podían medir nada tal como viene el entorno, 31 (53 %) reflejan una carencia del entorno. No discriminar en este entorno no hace intrínsecamente inválida a una tarea.

4.2 Variación de una configuración sin cambios

Tabla 2*Dos corridas de la misma configuración sobre las mismas tareas*

Conjunto de tareas	Resueltas, corridas 1 y 2	Cambio neto del total	Tareas que cambian de resultado	Tareas que cambian de categoría de fallo
16 tareas de rich	4 y 4	0	2	—
15 tareas de rich	3 y 3	0	2	6
30 tareas (rich y fastapi)	6 y 7	1	1	13

Nota. Un par de corridas por fila; los denominadores son las tareas del conjunto. La raya indica que la categoría no se registró.

La Tabla 2 contiene dos medidas. Por tarea, 1 o 2 cambiaron de resultado entre corridas idénticas; el total de resueltas cambió en 0, 0 y 1. Tres pares no establecen una diferencia mínima detectable para ninguna de las dos. Entre sesiones la dispersión fue mayor: siete corridas base sobre las 15 tareas de rich compartidas resolvieron 2, 3 o 4, y hasta 3 tareas difirieron entre dos de ellas. La categoría de fallo de una tarea (sin parche, el parche no pasa las pruebas, presupuesto agotado) cambió en 13 de 30 tareas, frente a 1 de 30 para el resultado: en estos datos, la tabla de fallos de una sola corrida no fue una guía estable de qué arreglar.

En la tabla pública, el mismo archivo local enviado dos veces obtuvo 0,06 y 0,05. No pudimos verificar que los archivos almacenados fueran idénticos: no es una réplica controlada. Como fracciones truncadas de una tabla de 58 tareas, el menor tamaño compatible con las notas publicadas, son 4 y 3 tareas; esa lectura es nuestra.

4.3 Seis intervenciones, ninguna mejora establecida

Tabla 3

Cada cambio frente a la línea base en las mismas tareas

Cambio	Predicción anotada antes de la corrida	Resueltas: base, cambio	Veredicto sobre la predicción
Razonamiento encendido, límite de 4 minutos	Sin ganancia	4 y 4, 3 de 16	Cumplida, para ese presupuesto
100 llamadas a herramientas en vez de 40	Más tareas resueltas	3 y 3, 2 de 15	Refutada
Diseño público de dos etapas (localizador de solo lectura y editor)	Más tareas resueltas	3 y 3, 3 de 15	Refutada
20 llamadas a herramientas	Al menos 2 resueltas	2, 2 de 15	Cumplida
Regla escrita contra las llamadas repetidas	No adelanta la primera edición	2, 4 de 15	Cumplida; ganancia neta de dos, no replicada
Regla de una línea para la herramienta de edición	Las sesiones atrapadas bajan a 1 o menos	6 y 7, 7 de 30	Cumplida en el mecanismo; mismas tareas resueltas

Nota. Una corrida por cambio. Los conteos base son de una o dos corridas, según se muestra. «Atrapada» significa tres o más llamadas de edición mal formadas seguidas.

Ninguna fila de la Tabla 3 establece una mejora, lo que no es evidencia de que no haya efecto: el mayor aumento neto (de 2 a 4) fue una sola corrida sobre 15 tareas. Con razonamiento encendido y límite de 4 minutos, 11 de 13 sesiones no resueltas se quedaron sin tiempo; esa fila describe el razonamiento bajo ese presupuesto y no dice nada de uno mayor. Una séptima condición, con tres ajustes cambiados a la vez (razonamiento encendido, 60 llamadas, 4,5 minutos), no tiene resultado a la fecha de corte.

La última fila es nuestra observación más sólida, porque separa un mecanismo de un resultado. El agente llamó a la herramienta de edición sin su argumento obligatorio en 107 de 134 llamadas en una corrida base y 49 de 75 en la otra, y repitió la misma llamada fallida: 3 de 30 sesiones quedaron atrapadas en cada corrida. Una llamada sintética cuyo argumento de texto se cierra con el delimitador equivocado reproduce los mismos argumentos mal formados en el analizador público del servidor del modelo; la salida cruda del modelo no se guardó. En la única corrida con una línea de instrucción añadida el fallo no ocurrió (0 de 26 llamadas, 0 sesiones atrapadas). Las tareas resueltas fueron las mismas 7 que en la segunda corrida base, 6 de ellas resueltas también en la primera. Las tareas atrapadas en una corrida base no se resolvieron en la otra, aunque 2 de las 3 no quedaron atrapadas allí.

De las 40 predicciones anotadas, 30 se evaluaron: 19 se cumplieron y 11 se refutaron. Cuatro no se pudieron evaluar o nunca se ejecutaron, y seis están pendientes.

4.4 Dónde se pierde el agente

Veintitrés de las 30 tareas nunca se resolvieron en tres corridas (69 sesiones). Quince tienen un enunciado de menos de 250 caracteres: un título y, en la mayoría, una referencia a una incidencia externa. Ninguna tarea de enunciado corto se resolvió: 0 de 15 en cada corrida de 30 tareas, y 0 de 16 contando una más del conjunto anterior de 16. El umbral se eligió tras ver los datos de rich y se sostuvo en una predicción anotada antes de correr por primera vez las de fastapi (0 de 6 cortas, 3 de 9 largas).

Un revisor leyó el enunciado, el parche de referencia, las pruebas ocultas y el código circundante de las 23 tareas y juzgó 12 resolubles con el enunciado y el repositorio, 9 resolubles solo adivinando un nombre, un mensaje o un valor no dicho, y 2 no resolubles. Este juicio es retrospectivo, con acceso al parche de referencia y a las pruebas ocultas; no es una medida independiente de dificultad.

Tabla 4

Etapas más lejanas alcanzadas en al menos dos de tres corridas

	Etapas más lejanas	23 nunca resueltas	de ellas, 12 juzgadas resolubles	7 resueltas
No lee el archivo que había que cambiar	10	6	0	0
Lo lee y no lo edita	6	4	0	0
Edita otra región del archivo	2	0	0	0
Llega a la región correcta	5	2	7	7

Nota. Las columnas cuentan tareas, ubicadas en la etapa más alta alcanzada en dos o más de tres corridas.

Diez de las 12 tareas juzgadas resolubles no llegaron a una edición pertinente en al menos dos de tres corridas (Tabla 4). De las 69 sesiones sobre tareas nunca resueltas, 25 nunca intentaron editar; en las 44 que sí, el primer intento llegó tras una mediana del 65 % del presupuesto de llamadas, frente al 26 % en las 20 sesiones resueltas. La mitad de sus llamadas (1267 de 2532) repitió una llamada idéntica anterior, y 36 de las 69 sesiones hicieron 40

llamadas o más, mientras 5 terminaron con el error de tiempo agotado del arnés (3 de ellas entre las 36). Otros dos defectos de herramienta gastan llamadas: la de lectura perdió su rango de líneas en 344 de 851 llamadas en las tres corridas, tanto en sesiones resueltas como fallidas, y la búsqueda por similitud no devolvió nada en las 221 llamadas.

Cuando hay parche, rara vez está cerca: de las 41 de esas sesiones con parche, 16 contienen solo guiones de depuración, pruebas propias del agente o documentación, y en 23 de las otras 25 el agente no pasa ninguna de las pruebas objetivo. Ningún fallo de verificación se debió al entorno.

5. Amenazas a la validez

- **Muestra pequeña.** Treinta tareas, dos repositorios, tres corridas; una corrida para la regla de la herramienta de edición.
- **Exploratorio.** El diseño prerregistrado no se ejecutó, las tareas de desarrollo se reutilizaron y la regla operativa no tiene prueba ni análisis de potencia.
- **Juicio.** La resolubilidad es una sola lectura retrospectiva. Los análisis de trazas, pruebas y resolubilidad los produjeron asistentes basados en modelos de lenguaje bajo la dirección del autor, sin réplica independiente.
- **Lo público no es lo oculto.** Los organizadores declaran que las tareas ocultas se curaron aparte; nada aquí predice la nota oculta.

6. Reproducibilidad y uso de datos

Fecha de corte: 7 de octubre de 2026, 01:20 UTC. La configuración base tiene la huella 3b0e87556166 y la variante de la herramienta de edición 7e2082197834: los primeros 12 dígitos hexadecimales del SHA-256 de las codificaciones base64 de los seis archivos de configuración, ordenadas como cadenas y concatenadas sin nombres ni separadores. Los guiones de validez y el lector de la tabla pública son públicos:

https://github.com/cherrera0001/Agents_Learning_Loops. La competición prohíbe redistribuir el contenido de las tareas: no se publican resultados por tarea ni trazas.

7. Conclusión

Entre corridas idénticas cambiaron de resultado 1 o 2 tareas, el total cambió como máximo en uno dentro de un par y de 2 a 4 de 15 entre siete corridas base, y la categoría de fallo cambió en 13 de 30 tareas; no leímos diferencias de ese tamaño como efectos, y no afirmamos ningún umbral de detección. Cincuenta y ocho de 129 tareas públicas no podían medir nada en el entorno tal como viene. En seis cambios de configuración no establecimos ninguna mejora. En una corrida, un fallo de herramienta no ocurrió y las tareas resueltas fueron las mismas 7 que en la segunda corrida base. En 16 de las 23 tareas nunca resueltas, el agente no editó el archivo que había que cambiar en al menos dos de tres corridas.

Referencias

- Aleithan, R., Xue, H., Mohajer, M. M., Nnorom, E., Uddin, G., y Wang, S. (2024). *SWE-Bench+: Enhanced coding benchmark for LLMs*. arXiv. <https://doi.org/10.48550/arXiv.2410.06992>
- Bjarnason, B. H., Silva, A., y Monperrus, M. (2026). *On randomness in agentic evals*. arXiv. <https://doi.org/10.48550/arXiv.2602.07150>
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- Gloaguen, T., Mündler-Sasahara, N., Müller, M. N., Raychev, V., y Vechev, M. (2026). *Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?* arXiv. <https://doi.org/10.48550/arXiv.2602.11988>
- Gluzdov, D. (2026). *Gemma 4: Measure before you tune* (Versión 7, última ejecución el 1 de octubre de 2026) [Cuaderno computacional]. Kaggle. Recuperado el 4 de octubre de 2026, de <https://www.kaggle.com/code/dmitriigluzdov/gemma-4-measure-before-you-tune>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., y Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? En *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
- Liang, S., Garg, S., y Moghaddam, R. Z. (2026). The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. En *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice* (pp. 395–405). Association for Computing Machinery. <https://doi.org/10.1145/3786583.3786882>

- Miller, E. (2024). *Adding error bars to evals: A statistical approach to language model evaluations*. arXiv. <https://doi.org/10.48550/arXiv.2411.00640>
- Nosek, B. A., Ebersole, C. R., DeHaven, A. C., y Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11), 2600–2606. <https://doi.org/10.1073/pnas.1708274114>
- Wang, Y., Pradel, M., y Liu, Z. (2026). Are "solved issues" in SWE-bench really solved correctly? An empirical study. En *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering* (pp. 169–181). Association for Computing Machinery. <https://doi.org/10.1145/3744916.3764576>
- Xodarev, A. (2026). *119 of 129 sound: The Gemma 4 grader, rebuilt* (Versión 22, última ejecución el 1 de octubre de 2026) [Cuaderno computacional]. Kaggle. Recuperado el 4 de octubre de 2026, de <https://www.kaggle.com/code/busyaprime/119-of-129-sound-the-gemma-4-grader-rebuilt>
- Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., Liu, S., Cui, S., Kapoor, S., Longpre, S., Meng, K., Weiss, R., Barez, F., Gupta, R., Dhamala, J., Merizian, J., Giulianelli, M., Coppock, H., Ududec, C., Kellermann, A., Sekhon, J., . . . Kang, D. (2025). Establishing best practices in building rigorous agentic benchmarks. En *Advances in Neural Information Processing Systems 38* (pp. 184435–184475). Neural Information Processing Systems Foundation. <https://doi.org/10.52202/085713-5547>